LAB ASSIGNMENT 8
Trees: BST and Traversing algorithms

1. Write a program to implement Binary Search Tree (Insertion, Deletion, traversing as In-order, Pre-order and Post-order).

Screenshot captured
You can paste the image from the clipboard.

```c
#include <stdio.h>
#include <stdlib.h>
struct Node{
    int data;
    struct Node* left;
    struct Node* right;
};
struct Node* createNode(int data){
    struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
    if (newnode==NULL){
        printf("problem in creation of newnode\n");
        exit(1);
    }
    newnode->data=data;
    newnode->left=NULL;
    newnode->right=NULL;
    return newnode;
}
struct Node* insert(struct Node*root,int data){
    if (root==NULL){return createNode(data);}
    if (data<root->data){
        root->left=insert(root->left,data);
    }else if (data>root->data){
        root->right=insert(root->right,data);
    }
    return root;
}
struct Node* findMin(struct Node*node){
    struct Node*curr=node;
    while(curr&&curr->left!=NULL){
        curr=curr->left;
```

NORMAL master assignment8_q1.c utf-8

```c
struct Node* findMin(struct Node*node){
    struct Node*curr=node;
    while(curr&&curr->left!=NULL){
        curr=curr->left;
    }
    return curr;
}
struct Node* deleteNode(struct Node*root,int data){
    if (root==NULL){return root;}
    if (data<root->data){
        root->left=deleteNode(root->left,data);
    }
    else if (data>root->data){
        root->right=deleteNode(root->right,data);
    }
    else {
        if (root->left=NULL){
            struct Node* temp=root->right;
            free(root);
            return temp;
        }
        else if (root->right==NULL){
            struct Node*temp=root->left;
            free(root);
            return temp;
        }
        struct Node*temp=findMin(root->right);
        root->data=temp->data;
        root->right=deleteNode(root->right,temp->data);
    }
    return root;
```

NORMAL master assignment8_q1.c utf-8

```c
}
void inOrder(struct Node* root){
    if (root!=NULL){
        inOrder(root->left);
        printf("%d ",root->data);
        inOrder(root->right);
    }
}
void PreOrder(struct Node*root){
    if (root!=NULL){
        printf("%d",root->data);
        PreOrder(root->left);
        PreOrder(root->right);
    }
}
void PostOrder(struct Node*root){
    if (root!=NULL){
        PostOrder(root->left);
        PostOrder(root->right);
        printf("%d ",root->data);
    }
}
int main (){
    struct Node*root=NULL;
    printf("Inserting nodes: 50, 30, 70, 20, 40, 60, 80\n");
    root=insert(root,50);
    insert(root,30);
    insert(root,70);
    insert(root,20);
    insert(root,40);
    insert(root,60);
```

```c
int main (){
    struct Node*root=NULL;
    printf("Inserting nodes: 50, 30, 70, 20, 40, 60, 80\n");
    root=insert(root,50);
    insert(root,30);
    insert(root,70);
    insert(root,20);
    insert(root,40);
    insert(root,60);
    insert(root,80);
    printf("\nIn-order traversal: ");
    inOrder(root);
    printf("\nPre-order traversal :");
    PreOrder(root);
    printf("\nPost-order traversal :");
    PostOrder(root);
    printf("\n\nDeleting 20 (node with no children)...\n");
    root = deleteNode(root, 20);
    printf("In-order traversal after deletion: ");
    inOrder(root);

    printf("\n\nDeleting 70 (node with one child)...\n");
    root = deleteNode(root, 70);
    printf("In-order traversal after deletion: ");
    inOrder(root);

    printf("\n\nDeleting 50 (root with two children)...\n");
    root = deleteNode(root, 50);
    printf("In-order traversal after deletion: ");
    inOrder(root);

```

```
~/Documents/dslab  master ?2  g++ assignment8_q1.c
~/Documents/dslab  master ?2  ./a.out
Inserting nodes: 50, 30, 70, 20, 40, 60, 80

In-order traversal: 20 30 40 50 60 70 80
Pre-order traversal :50302040706080
Post-order traversal :20 40 30 60 80 70 50

Deleting 20 (node with no children)...
In-order traversal after deletion: 30 40 50 60 70 80

Deleting 70 (node with one child)...
In-order traversal after deletion: 30 40 50 80

Deleting 50 (root with two children)...
In-order traversal after deletion: 80
~/Documents/dslab  master ?2  nvim assignment8_q1.c
```